

Programming 3A

PROG7311 A1

Narishka Sewraj

ST10433872

Executive summary and introduction :

TechMove Logistics currently works on fragmented legacy systems such as disjointed spreadsheets, emails and manual phone calls. This process has many consequences including distribution data, bottlenecks, expired contracts and lost invoices. This proposal of Global Management System (GLSM) will resolve many of these issues caused by the legacy systems by putting into place an enterprise platform that is scalable and containerised. An outline of more efficient software will be constructed in this proposal for the TechMove Board of directors. This new system will allow for scalability, strengthen compliance and automated workflows.

Enterprise framework strategy :

TOGAF is chosen for the following reasons :

An Enterprise Architecture framework known as The Open Group Architecture Framework aids larger scale organisations plan and create an IT infrastructure. TOGAF allows enterprises to control their technology infrastructure efficiently and effectively by aligning their business goals and IT strategy. TOGAF is made up of many principles, the best actions for developing and maintaining and guidelines for enterprise architecture. It supports flexibility, compatibility and consistency in IT systems. An organization can better understand how well its infrastructure will do to achieve business objectives when using TOGAF to model and analyze its architecture (Ardoq, 2024). Risk management and governance are integrated when using TOGAF, this will help organizations in controlling risks and finding risks with regard to IT architecture (Ardoq, 2024). Continuous evaluation of TOGAF makes sure that it stays relevant and can adapt when needed (TheKnowledgeAcademy, n.d.). This makes it perfect for TechMove logistics company as it will resolve many of the current issues the company faces.

TOGAF Architecture Development Method :

Design pattern selection :

Builder pattern :

The builder pattern is a creational design pattern, it allows you to create contract objects step by step as it separates the construction of a complex object from its representation. This is crucial if an object requires variants or many configurations (GeeksforGeeks, 2023). A validation point is created, this will make sure that all data that is required of the user such as pricing and client details are properly set and valid before a contract is created. By enforcing business rules this resolves any validation issues and makes sure that system contains legit and valid contracts only. By isolating the construction logic from the contract object the builder pattern also facilitates decoupling. Therefore, making the system easier to maintain and scale up and reducing dependencies.

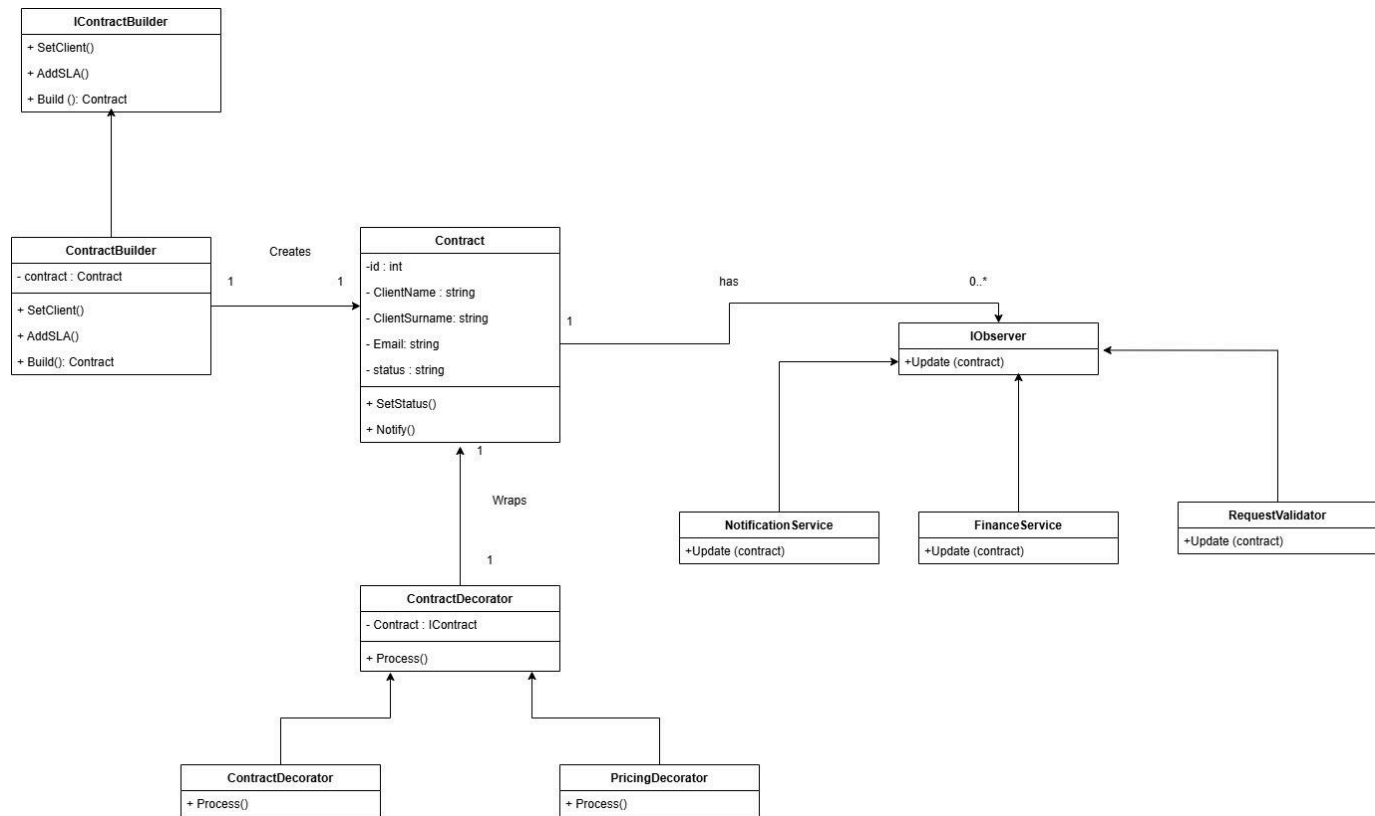
Observer pattern :

The observer pattern is a behavioral design pattern that allows multiple objects to be notified about events that may occur to the object that is being observed (Refactoring Guru, 2014). This pattern will ensure loosely coupled architecture and make sure that validation occurs in the GLMS over many components. The observer pattern is an event driven architecture and is useful for TechMove as they need parts of the system to behave differently when a major change occurs in status such as active/expired. The contract will act as the publisher and the other parts such as, notifications and service requests will act as subscribers there will have real time updates. The observer pattern removes direct dependencies. It is not necessary for the contract to know what sections are affected, it will only send a notification. More observers can be joined in at a later stage without changing the contract class as each observer will only handle its own logic. This will allow the system to scale up and be flexible. This pattern makes sure that there is consistent validation as rules are applied in the real time as well as a loosely coupled system, with regard to the compliance and workflow issues that TechMove faces.

Decorator pattern :

This pattern is a structural pattern, without changing an object's code it adds a new behaviour/responsibility to it. Extra functionalities are layered around an object (GeeksforGeeks, 2023). By enabling any contract feature (such as compliance, price, and insurance) to be implemented as a distinct decorator that does its own validation tests, the decorator pattern resolves validation difficulties. This keeps contracts from being incomplete by ensuring that rules are put into place and applied prior to contract acceptance.

By separating responsibilities from the main contract, it also resolves decoupling problems. Each decorator is introduced separately and concentrates on a single function, as opposed to one big overloaded class containing all logic. As a result, the system is more adaptable, modular, and simple to manage.



Non functional requirements and scalability strategy :

Availability requirements :

Even when a system is being maintained, facing issues or failures it should still remain fully operational for users, this is known as availability (Huynh, 2024). This is important for TechMove as they could lose clients if they do not have availability. GLMS should be able to operate even when there are failures, global access should be up and running at all times and there should be at least 99.9 % uptime.

Interoperability requirements :

Allowing different IT systems such as GLMS to interact and share functionally with the least amount of human interaction is known as interoperability (Lindermulder, 2025). Cross platform communication of services should be allowed, integration with external API's and it should be able support standard data formats. During the transition of systems GLMS should be compatible with the legacy system.

Scalability Strategies:

If there is a huge surge in contracts the architecture should be able to handle this without any issues.

Horizontal scaling :

Horizontal scaling is when an application automatically scales by adding or taking away nodes as the workload gets heavier or lighter (Virtana, n.d.). In this case more services can be added instead of increasing the servers. Horizontal scaling can be implemented by containerization technologies such as Docker, to package microservices. Automated scaling and deployment can be made possible with orchestration tools such as Kubernetes, it makes taking control of complex distributed systems much easier (GeeksforGeeks, 2024).

Load Balancing :

A single node will not be overloaded as load balancers built into cloud platforms(Cloud Load Balancing in GCP) such as AWS/Microsoft Azure will distribute traffic accordingly across many microservices (GeeksforGeeks, 2024) . If a service has an issue the load balancer can automatically reroute traffic to a working service, in this event there should also be redundant services so if one fails another can continue in its place (Huynh, 2024). Some load balancing algorithms can be put into place such as the Round robin method or IP hash method (Amazon Web Services, 2024).

Caching :

Data that is used often is collected and stored in a transient data store, this is known as cache, this data is then pulled when the application requests it. The data can be stored within a file, database or memory (CODE, 2024). The cache servers are scattered among several nodes or computers in a distributed caching system, enabling scalability and enhanced performance. Distributed caching is mostly used to keep frequently accessed data in memory instead of retrieving it from a remote database or the hard drive. Because retrieving data from memory is quicker than disc access, this increases the speed of data access. If there is a surge in contracts additional cache servers can be added to a distributed cache system without disturbing ongoing operations. Some distributed caching systems that can be used in this scenario include Redis, NCache and Memcached (CODE, 2024).

Conclusion :

In summary, the suggested GLMS offers TechMove a scalable, dependable, and effective solution. Through horizontal scalability, load balancing, and caching, it facilitates smooth service integration, increases system availability, and allows quick expansion. This guarantees that the system can manage pressure without lacking in performance, making it the best choice for TechMove's future growth.

Reference list :

Amazon Web Services (2024). What is Load Balancing? Load Balancing Explained - AWS. [online] Amazon Web Services, Inc. Available at: <https://aws.amazon.com/what-is/load-balancing/> [Accessed 26 Mar. 2026].

Ardoq (2024). What is TOGAF? Benefits of a Customizable EA Framework. [online] Ardoq.com. Available at: <https://www.ardoq.com/knowledge-hub/togaf> [Accessed 24 Mar. 2026].

CODE. (2024). Distributed Caching: Enhancing Scalability and Performance in ASP.NET 8 Core. [online] Available at: <https://www.codemag.com/Article/2403081/Distributed-Caching-Enhancing-Scalability-and-Performance-in-ASP.NET-8-Core> [Accessed 26 Mar. 2026].

GeeksforGeeks (2023). Gang of Four (GOF) Design Patterns. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/gang-of-four-gof-design-patterns/> [Accessed 24 Mar. 2026].

GeeksforGeeks (2024). How to Scale any System Horizontally? [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/how-to-scale-any-system-horizontally/> [Accessed 26 Mar. 2026].

Hornford, D. (2025). TOGAF Architecture Development Method Phases Explained. [online] Conexiam. Available at: <https://conexiam.com/togaf-adm-phases-explained/#a> [Accessed 25 Mar. 2026].

Huynh, D. (2024). System Availability - Duy Huynh - Medium. [online] Medium. Available at: <https://medium.com/@vndee.huynh/system-design-note-03-availability-dbbb78e7853a> [Accessed 26 Mar. 2026].

Lindermulder, G. (2025). Interoperability. [online] Ibm.com. Available at: <https://www.ibm.com/think/topics/interoperability>.
Nortal.com. (2024). Strategies promoting interoperability across systems. [online] Available at: <https://nortal.com/insights/strategies-promoting-interoperability-across-systems> [Accessed 26 Mar. 2026].

Refactoring Guru (2014). Observer. [online] Refactoring.guru. Available at: <https://refactoring.guru/design-patterns/observer> [Accessed 24 Mar. 2026].

Sourabh-Hajela (2009). Mastering TOGAF: An Introduction To The TOGAF Framework And ADM Phases (Part 1 Of 3) - CIO Portal. [online] CIO Portal. Available at: <https://cioindex.com/reference/part-1-introducing-the-open-group-architecture-framework-togaf/> [Accessed 25 Mar. 2026].

Virtana. (n.d.). What is horizontal scaling (scaling out)? [online] Available at: <https://www.virtana.com/glossary/what-is-horizontal-scaling/> [Accessed 26 Mar. 2026].

www.boc-group.com. (n.d.). Demystifying TOGAF: A Guide to Streamlining Architecture Processes. [online] Available at:
<https://www.boc-group.com/en/blog/ea/ea-services-making-togaf-more-lightweight/>
[Accessed 25 Mar. 2026].